

SESSION 13 — Recursion and Scope

In this session we will learn two important ideas in programming

Recursion

Scope of variables

These concepts help us understand how programs execute internally.

Topics covered in this session

Recursion Recursive functions Function call stack Local scope Global scope Lifetime of variables

What is Recursion

Recursion means a function calling itself.

In simple words

A problem is solved by breaking it into smaller versions of the same problem.

Example idea

Suppose you want to count down from 5

You can think like this

5 → ask the same process to handle 4

4 → ask the same process to handle 3

3 → ask the same process to handle 2

2 → ask the same process to handle 1

This repeated self calling is recursion.

Important rule

Every recursive function must have a stopping condition. Otherwise the function will run forever.

Structure of a Recursive Function

Every recursive function has two parts

Base Case The condition where recursion stops.

Recursive Case The part where the function calls itself again.

General Structure

def function_name(parameters):

```
if base_condition:
    return result

else:
    return function_name(smaller_problem)
```

Example: countdown using recursion

```
def countdown(n):
```

```
    # Base case
    if n == 0:
        print("Done")
        return
```

```
    # Print current number
    print(n)
```

```
# Recursive call
countdown(n-1)

# Calling the function
countdown(5)

5
4
3
2
1
Done
```

✓ Example Problem — Factorial

Factorial means multiplying numbers from n down to 1.

Example

5 factorial = $5 \times 4 \times 3 \times 2 \times 1$

Which equals

120

Mathematically

$n! = n \times (n-1)!$

This formula itself suggests recursion.

```
def factorial(n):

    # Base case
    if n == 1:
        return 1

    # Recursive case
    return n * factorial(n-1)

# Testing the function
print("Factorial:", factorial(5))

Factorial: 120
```

$n = 5$

$5 * \text{fact}(4) \text{-----} n = 4 * \text{fact}(4) = 4 * \text{fact}(3) \text{----} n = 3 * \text{fact}(2) \text{-----} n = 2 * \text{fact}(1)$

✓ Another Example — Recursive Sum

Goal

Calculate the sum of numbers from 1 to n .

Example

sum(5)

$5 + 4 + 3 + 2 + 1$

```
def recursive_sum(n):

    # Base case
    if n == 1:
        return 1

    # Recursive case
    return n + recursive_sum(n-1)

print("Sum:", recursive_sum(5))
```

Sum: 15

```
def show_stack(n):

    print("Entering function with n =", n)

    if n == 0:
        return

    show_stack(n-1)

    print("Exiting function with n =", n)

show_stack(3)
```

```
Entering function with n = 3
Entering function with n = 2
Entering function with n = 1
Entering function with n = 0
Exiting function with n = 1
Exiting function with n = 2
Exiting function with n = 3
```

Scope of Variables

Scope means where a variable can be accessed.

Python mainly has two scopes

Local scope Global scope

Global Scope

A variable created outside a function is called a global variable.

It can be accessed anywhere in the program.

```
a = 5

print(a)

print("-----")

a = a-3

print(a)
print("-----")

a = a + 3

print(a)
print("-----")

a = a +1

print(a)
print("-----")

a = a - 4

print(a)
```

```
5
-----
2
-----
5
-----
6
-----
2
```

```
# Global variable
a = 10

def show_value():
    print("Inside function:", a)

show_value()

print("Outside function:", a)
```

```
Inside function: 10
Outside function: 10
```

```
# Global variable
a = 10

def show_value(a):
    a = 12
    a = a + 2
    print("Inside function:", a)

show_value(a)

print("Outside function:", a)
```

```
Inside function: 14
Outside function: 10
```

Local Scope

A variable created inside a function is called a local variable.

It can only be accessed inside that function.

```
def demo():

    y = 20    # local variable

    print("Inside function:", y)

demo()
# print(y)

# y cannot be accessed outside the function.
```

```
Inside function: 20
```

Lifetime of Variables

Lifetime means how long a variable exists in memory.

Global variables

Exist during the entire program.

Local variables

Exist only while the function is running.

Once the function finishes, local variables are destroyed.

```
def example():

    temp = 5

    print("Inside function:", temp)

example()

# temp no longer exists outside the function
# print(temp)
```

Inside function: 5

Summary

Recursion A function calling itself.

Base case Stops recursion.

Recursive case Breaks problem into smaller version.

Function call stack Keeps track of function execution.

Scope Defines where a variable can be used.

Global variables Accessible everywhere.

Local variables Accessible only inside the function.

Recursion and Scope Practice Questions

1. Write a recursive function called countdown that takes a number n and prints numbers from n down to 1.
2. Write a recursive function called print_numbers that prints numbers from 1 to n.
3. Write a recursive function called factorial that calculates the factorial of a number.
4. Write a recursive function called sum_numbers that calculates the sum of numbers from 1 to n.
5. Write a recursive function called power that takes two numbers base and exponent and returns base raised to the power exponent.
6. Create a global variable called x with value 10.
Write a function that prints the value of x inside the function.
7. Write a function where a variable y is created inside the function and printed.
Explain what happens if you try to print y outside the function.
8. Write a program where a global variable and a local variable have the same name.
Print both values to observe which one is used inside the function.
9. Write a recursive function called count_digits that returns the number of digits in a number.
10. Write a recursive function called recursive_sum_list that returns the sum of elements in a list.
11. Write a recursive function called reverse_number that reverses a number.
12. Write a recursive function called fibonacci that prints the nth Fibonacci number.

Start coding or [generate](#) with AI.

5
4
3
2
1

1
2
3
4
5

Factorial: 120

Sum: 15

Power: 8

Inside function: 10
Outside function: 10

Inside function: 20

```
Inside function: 20  
Outside function: 50
```

```
Digits: 5
```

```
List Sum: 100
```

```
Reversed number: 4321
```

```
Fibonacci number: 13
```

Start coding or [generate](#) with AI.